

UNIVERSITETET I OSLO

Lineær regresjon

Pierre Lison
Norsk Regnesentral (NR)

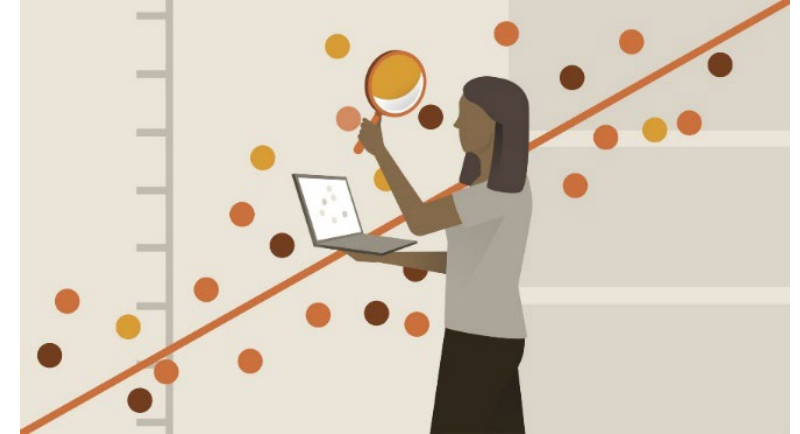
IN1160 – Introduksjon til
maskinlæring (Vår 2026)

18.02.2026



I dag skal vi snakke om:

Lineær regresjon



- En svært viktig modell innen statistikk og maskinlæring
- *Grunnlaget* for mange andre modeller, slik som logistisk regresjon (neste uke) og nevrale nettverk (om 2 uker)

Plan

- Modellen
- Læring
- Prediksjon
- Praktiske detaljer
- Oppsummering

Plan

- **Modellen**
- Læring
- Prediksjon
- Praktiske detaljer
- Oppsummering

Formål

- Lineær regresjon brukes til å predikere et tall y basert på et datapunkt x representert gjennom ulike trekk

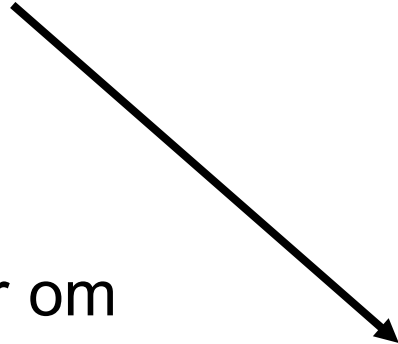
- Noen eksempler:



Gitt ulike opplysninger om en bolig (størrelse, sted, byggeår, osv.) som skal selges, hva blir *sluttprisen*?

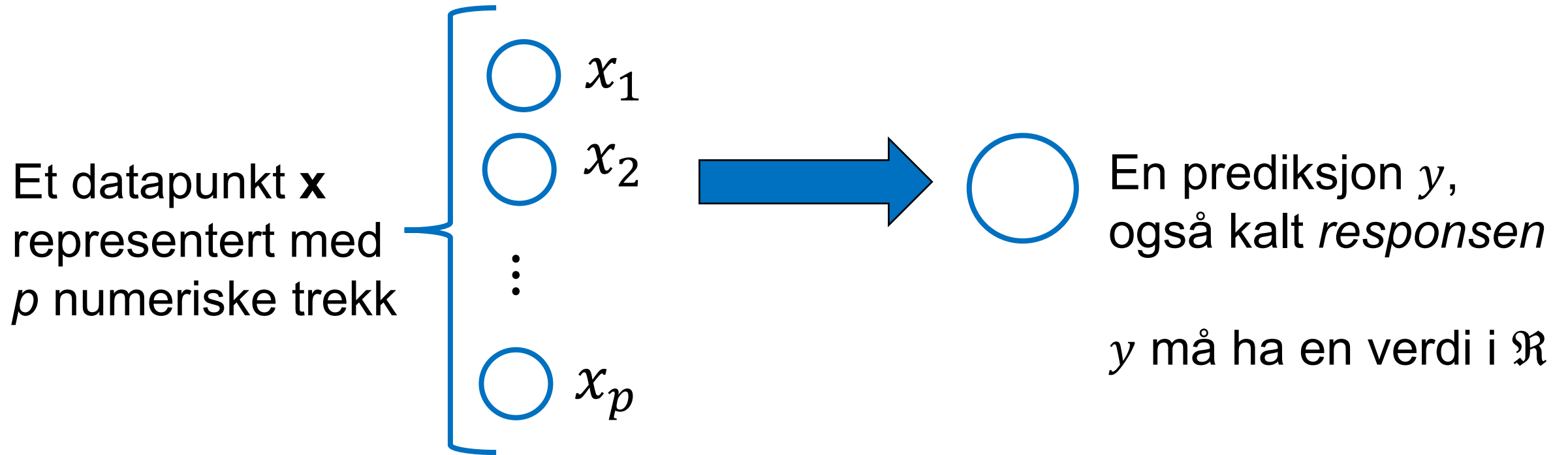


Gitt ulike opplysninger om en person (kjønn, yrke, kosthold, osv.), hva blir *personens forventede levealder*?

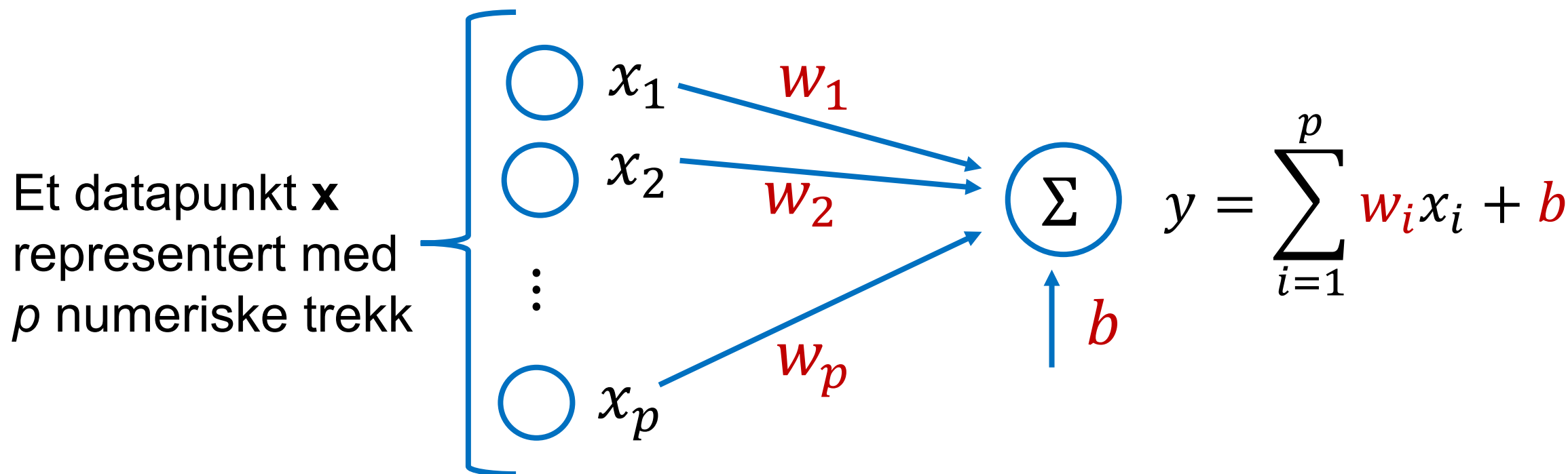


Gitt ulike trekk ekstrahert fra en nettside (tekstlengde, antall innkommende lenker, osv.), hva blir besøkstallet per måned ?

Input og output



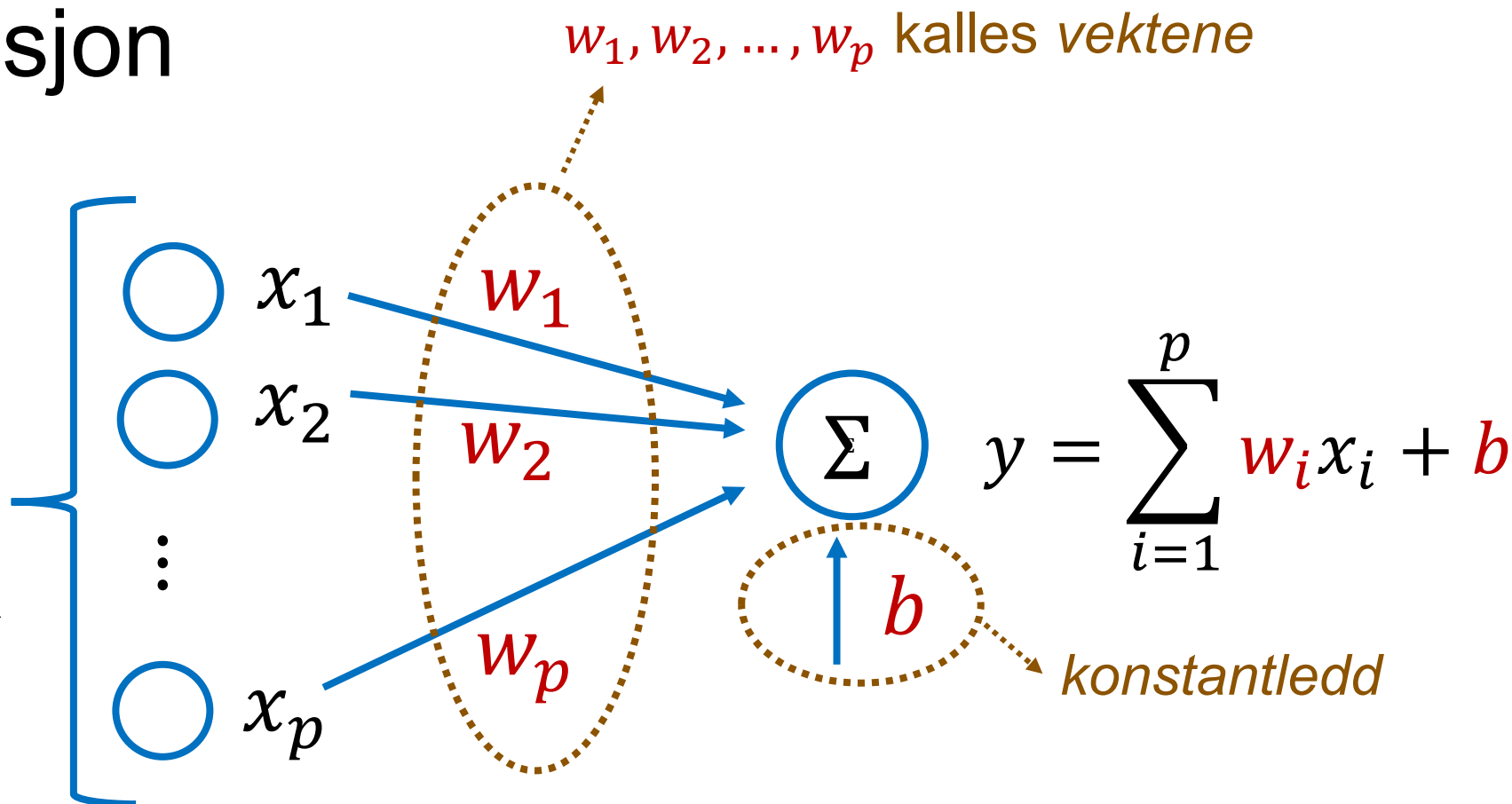
Lineær regresjon



En lineær regresjon defineres som en *vektet sum* av inputtrekkene + en konstant

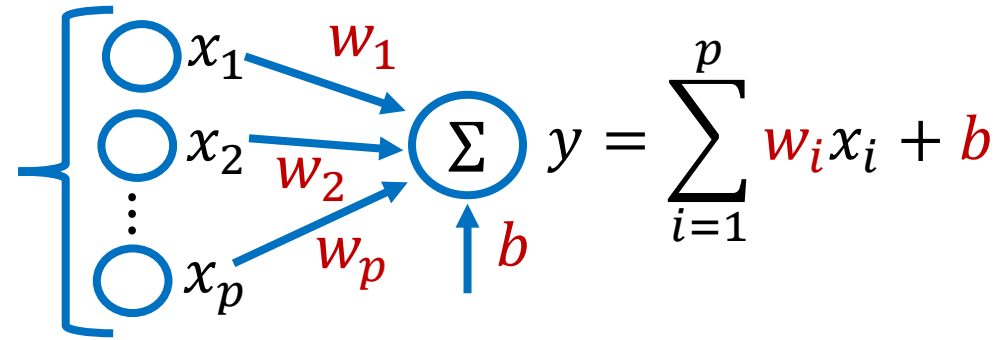
Lineær regresjon

Et datapunkt $\mathbf{x}$
representert med
 p numeriske trekk



Vektene $w_1, w_2, \dots, w_p$ og konstantleddet b utgjør modellens *parametere* (eller koefisienter) og er typisk estimert fra data

Mentimeter

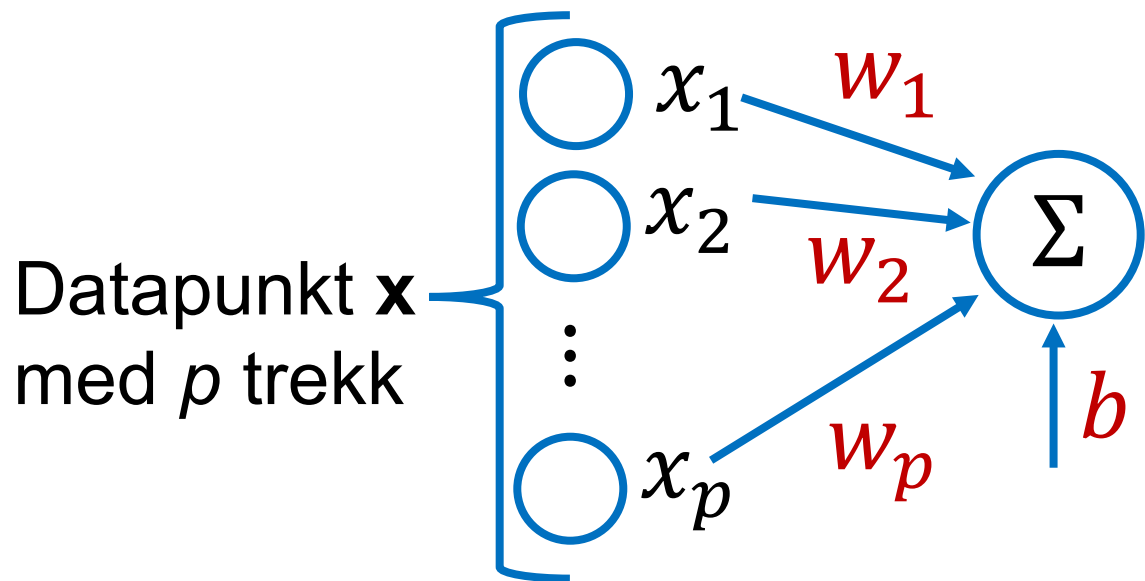


- La oss si vi ønsker å anslå boligpriser med en lineær regresjonsmodell basert på 2 trekk: *boligstørrelsen* (i kvm) og *antall baderom*
- Basert på historiske data har vi estimert en modell med følgende parametere:

$$w_{kvm} = 48 \quad w_{bad} = 405 \quad b = 306$$

- **Hva blir den predikerte prisen for en bolig på 126 kvm med 2 bad?**

Lineær regresjon i vektorform



$$y = \sum_{i=1}^p w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

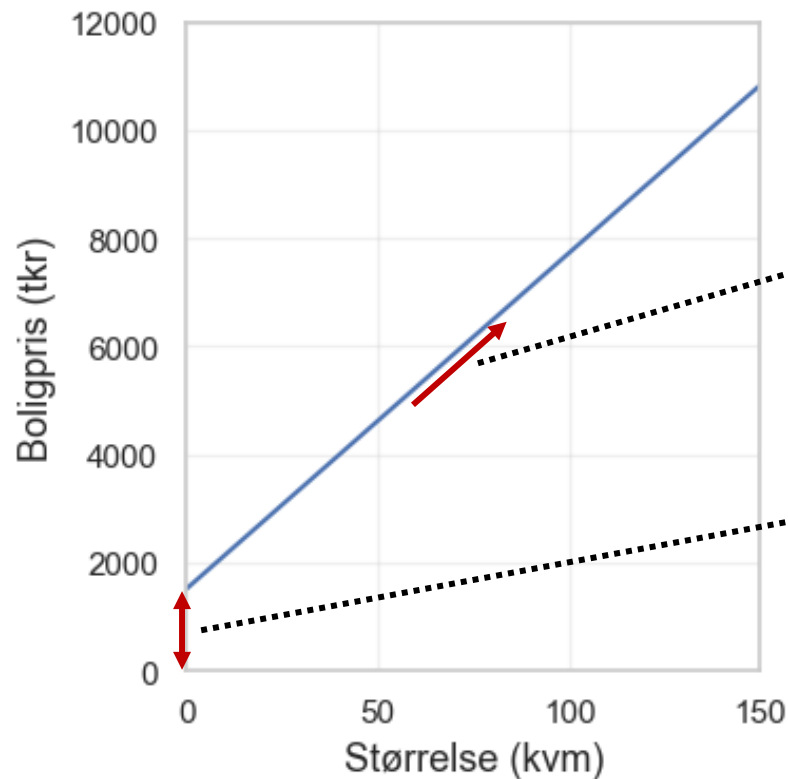
T = "transpose" (omgjør en kolonnevektor til en radvektor, eller omvendt)

hvor $\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \dots \\ w_p \end{bmatrix}$ og $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_p \end{bmatrix}$

Lineær regresjon kan skrives som prikkproduktet av en vektvektor $\mathbf{w}$ og en trekkvektor $\mathbf{x}$, + et konstantledd b

Hvorfor lineær regresjon ?

Når vi viser forholdet mellom ett enkelt trekk x og y får vi en *rett linje*:



Stigningstall = w
(hvor mye prisen y øker for hver ekstra kvm)

Konstantledd b
(verdien av y når $x = 0$. Derfor er b også kalt *skjæringspunktet*, eller *intercept* på engelsk)

Hvorfor lineær regresjon ?

★ 126 kvm med 2 bad

Med flere enn ett trekk i input $\mathbf{x}$ får vi en såkalt *hyperplan*:

Stigningstall w_2

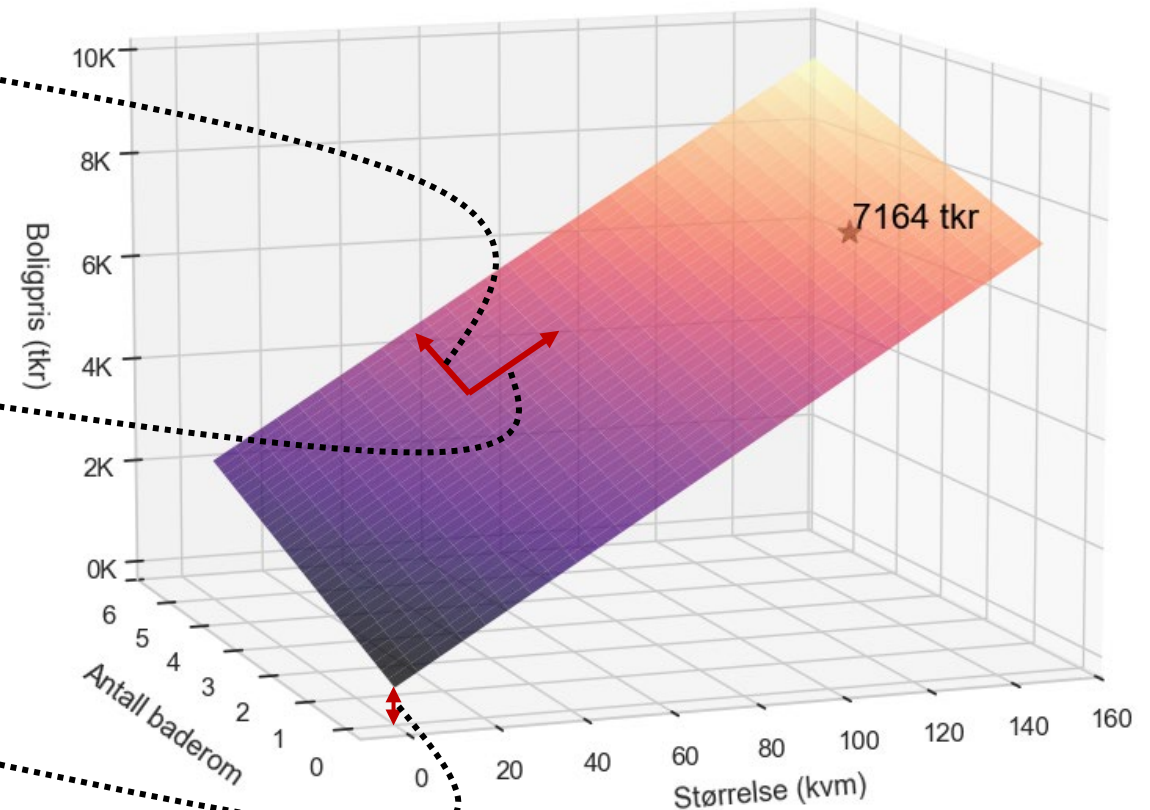
(hvor mye prisen øker for hvert nytt bad, gitt at antall kvm holdes konstant)

Stigningstall w_1

(hvor mye prisen øker for hver kvm, gitt at antall bad holdes konstant)

Konstantledd b

(verdien av y når alle trekkene $x_i = 0$)



Fordeler og ulemper



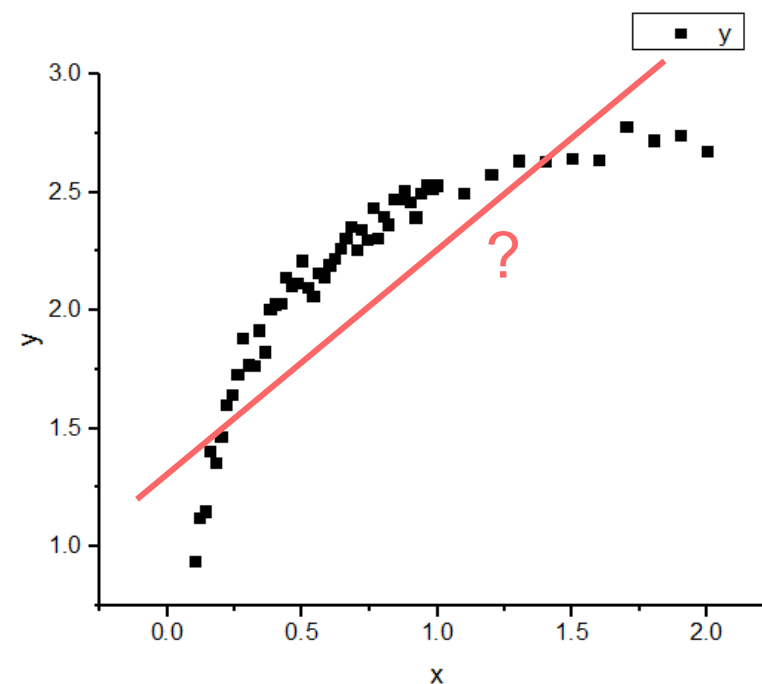
- Enkel og lett å tolke
- Rask å trene og å kjøre
- God som baseline



- Antar lineære sammenhenger mellom variabler

(**Merk:** lineær regresjon bygger også på andre statistiske antagelser utover linearitet, men vi skal ikke dekke disse i dette kurset)

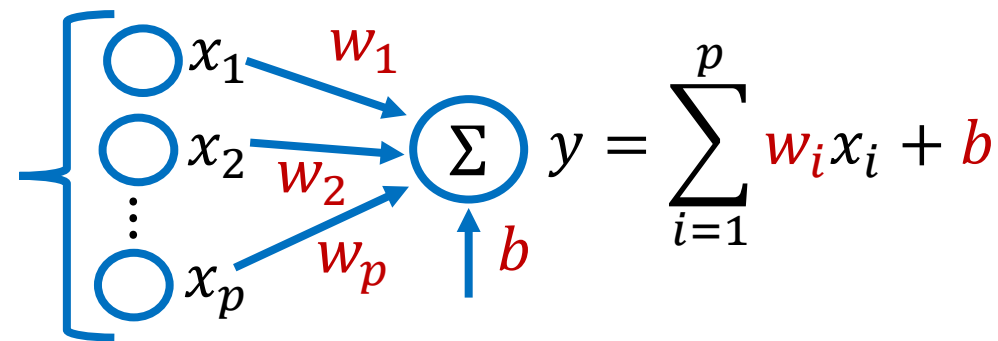
- Følsom for ekstremverdier (*outliers*)



Plan

- Modellen
- **Læring**
- Prediksjon
- Praktiske detaljer
- Oppsummering

Læring

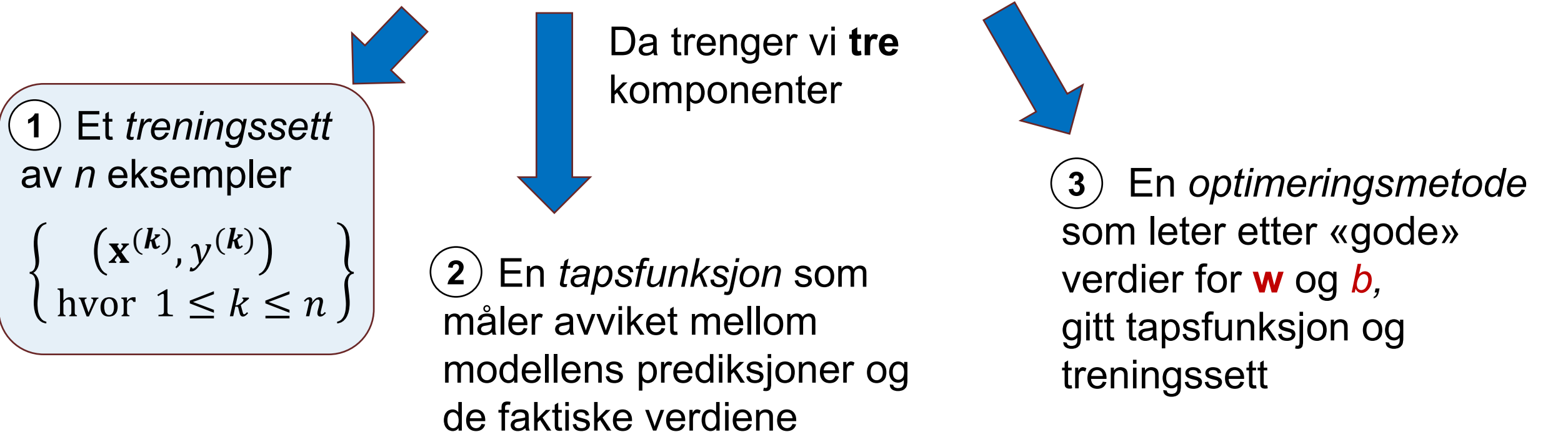


- Hvordan finner vi verdiene for parameterne $w_1, \dots, w_p$ og b ?
- Disse skal estimeres ut fra data (altså et *treningssett*)
- **Intuisjon:** læring handler om å finne verdiene for $w_1, \dots, w_p$ og b som gjør at “feilene” mellom hva modellen forutsier og fasitverdiene i treningssettet blir så *små som mulig*

→ Men hva betyr det *konkret* ? Det skal vi nå se nærmere på nå 😊

Læring

Intuisjon: læring handler om å finne verdiene for $w_1, \dots, w_p$ og b som gjør at “feilene” mellom hva modellen forutsier og fasitverdiene i treningssettet blir så *små som mulig*



① Et *treningssett* av n eksempler

$\left\{ \begin{array}{l} (\mathbf{x}^{(k)}, y^{(k)}) \\ \text{hvor } 1 \leq k \leq n \end{array} \right\}$

Da trenger vi **tre** komponenter

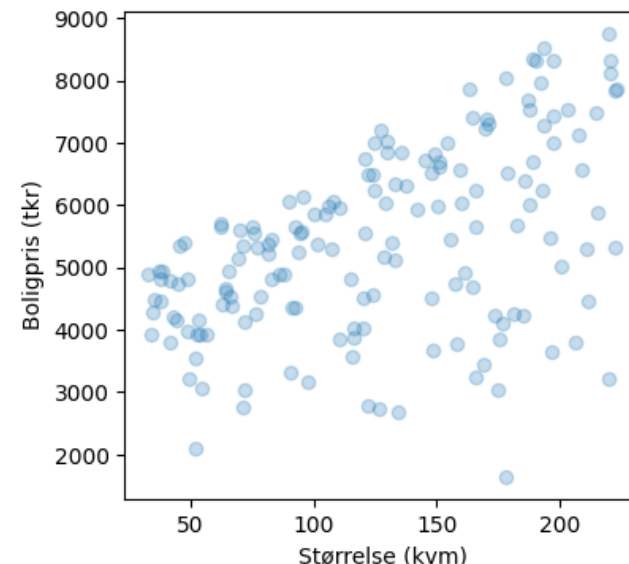
② En *tapsfunksjon* som måler avviket mellom modellens prediksjoner og de faktiske verdiene

③ En *optimeringsmetode* som leter etter «gode» verdier for $\mathbf{w}$ og b , gitt tapsfunksjon og treningssett

Treningssett

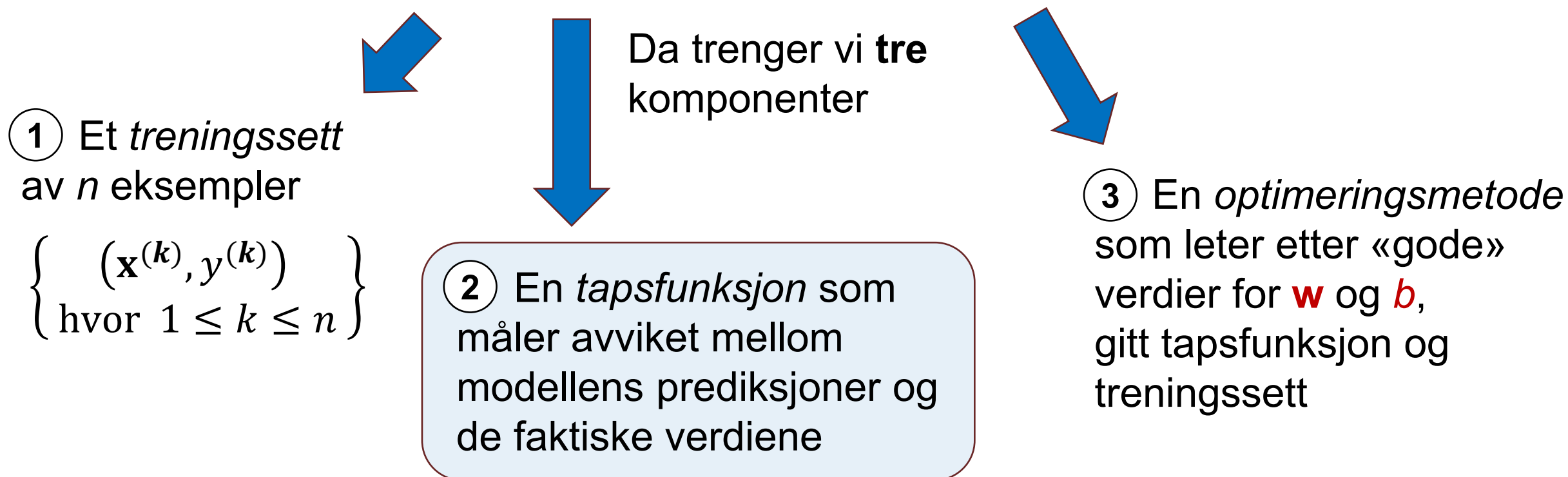
- Lineær regresjon må estimeres ut fra data
 - Akkurat som for k-NN (uke 3), bortsett fra at y er her et *numerisk tall* og ikke en klasse
- **Treningssett** = sett av n eksempler $\{(\mathbf{x}^{(k)}, y^{(k)})\}$ hvor $1 \leq k \leq n$
 - Altså eksempler av input $\mathbf{x}^{(k)}$ sammen med deres «ekte» output $y^{(k)}$
- Ofte er treningssettet ekstrahert fra historiske data
 - F.eks. kan output $y^{(k)}$ tilsvare den faktiske prisen boligen ble solgt for

Et treningssett med 50 eksempler
(her mer kun ett trekk: *boligstørrelsen*)



Læring

Intuisjon: læring handler om å finne verdiene for $w_1, \dots, w_p$ og b som gjør at “feilene” mellom hva modellen forutsier og fasitverdiene i treningssettet blir så *små som mulig*



Tapsfunksjon

- La $(\mathbf{x}^{(k)}, y^{(k)})$ være et eksempel fra treningssettet, hvor $\mathbf{x}^{(k)}$ er et bestemt input og $y^{(k)}$ er deres «fasitverdi»
- En **tapsfunksjon** (*loss function*) måler avviket mellom

den faktiske verdien $y^{(k)}$
fra treningssettet

og modellens prediksjon $\hat{y}^{(k)}$
regnet ut fra input $\mathbf{x}^{(k)}$:

Merk “hatten”, som brukes til å vise at dette er en
prediksjon fra modellen, ikke den faktiske verdien!

$$\hat{y}^{(k)} = \mathbf{w}^T \mathbf{x}^{(k)} + b$$

Tapsfunksjon

- En svært vanlig tapsfunksjon for lineær regresjon er å ta *kvadratet* av avviket mellom $\hat{y}^{(k)}$ og $y^{(k)}$:

$$l(\hat{y}^{(k)}, y^{(k)}) = (\hat{y}^{(k)} - y^{(k)})^2$$

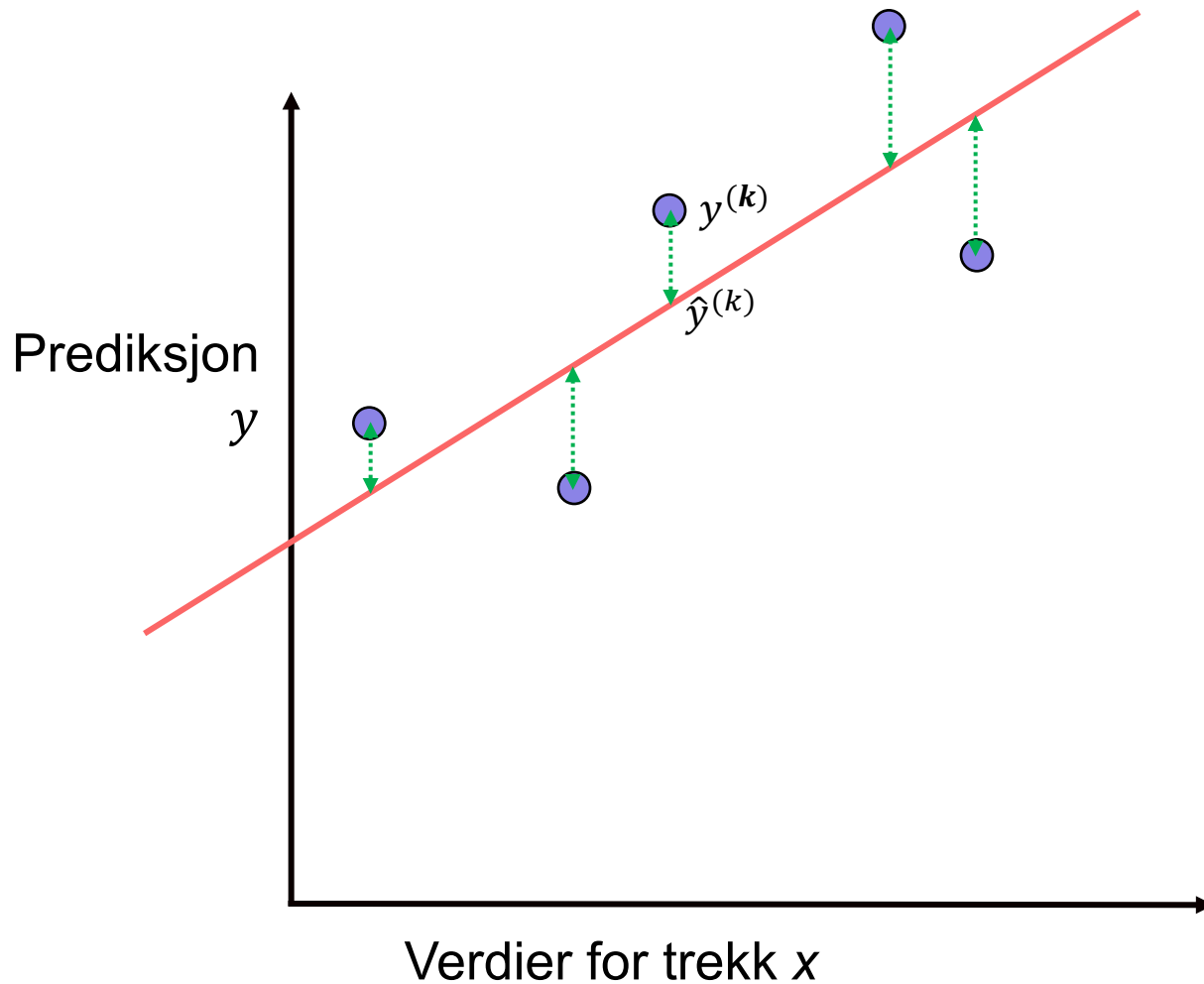
Hvorfor *kvadratet*? Det finnes flere grunner, men en viktig fordel er at det fanger opp avvik i begge retninger, altså både når $\hat{y}^{(k)} > y^{(k)}$ og omvendt.

- Vi kan deretter regne ut det gjennomsnittlige tapet på hele treningssettet, kalt **gjennomsnittlig kvadratfeil** eller *mean square error* (MSE) på engelsk:

$$\text{MSE} = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)}) = \frac{1}{n} \sum_{k=1}^n (\hat{y}^{(k)} - y^{(k)})^2$$

Merk: det finnes mange alternative tapsfunksjoner (og varianter med ulike typer «regulariseringer») i maskinlæring. Vi skal snakke litt mer om disse i forelesningen om «ML i praksis».

Tapsfunksjon



- For hvert punkt har vi et tap definert som $(\hat{y}^{(k)} - y^{(k)})^2$, altså kvadratet av avstandene vist i grønn
- *Gjennomsnittlig kvadratfeil (MSE):* gjennomsnitt over tapene for alle punktene i treningssettet
- Den “beste” regresjonsmodellen er den som minimiserer MSE

Merk: Denne estimeringsmetoden er flere hundre år gammel, og er kjent i statistikk som *minste kvadraters metoden*

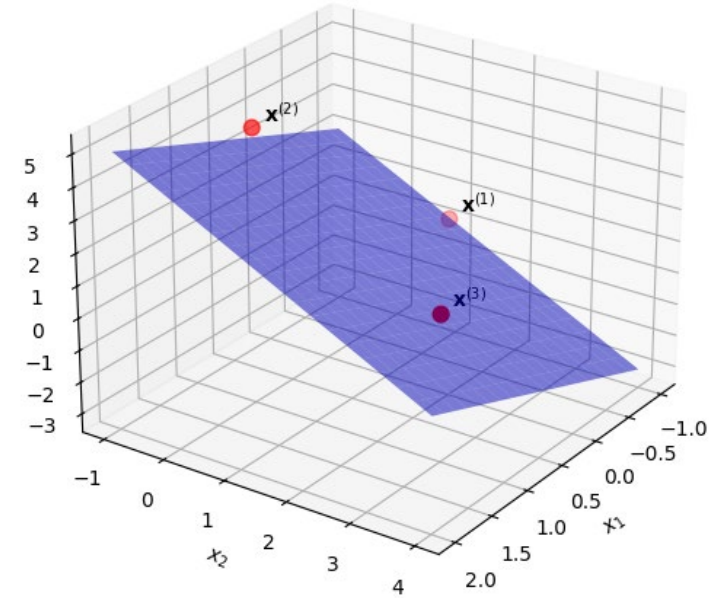
Mentimeter

Vi har et treningssett med 3 punkter:

- $\mathbf{x}^{(1)} = [0, 2]$ med $y^{(1)} = 2$
- $\mathbf{x}^{(2)} = [1, 0]$ med $y^{(2)} = 5$
- $\mathbf{x}^{(3)} = [1, 3]$ med $y^{(3)} = 1$

Og en lineær regresjonsmodell med $\mathbf{w} = [1, -1]$ og $b = 2$.

→ Hva er modellens
gjennomsnittlig kvadratfeil (MSE)
på disse tre datapunktene?

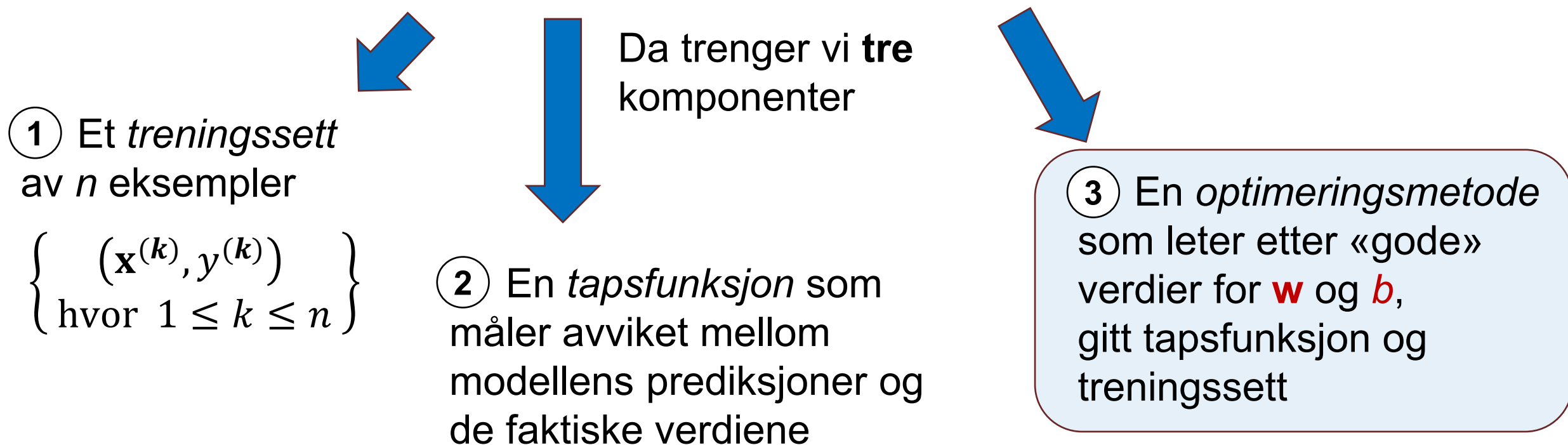


Noen nyttige formler:

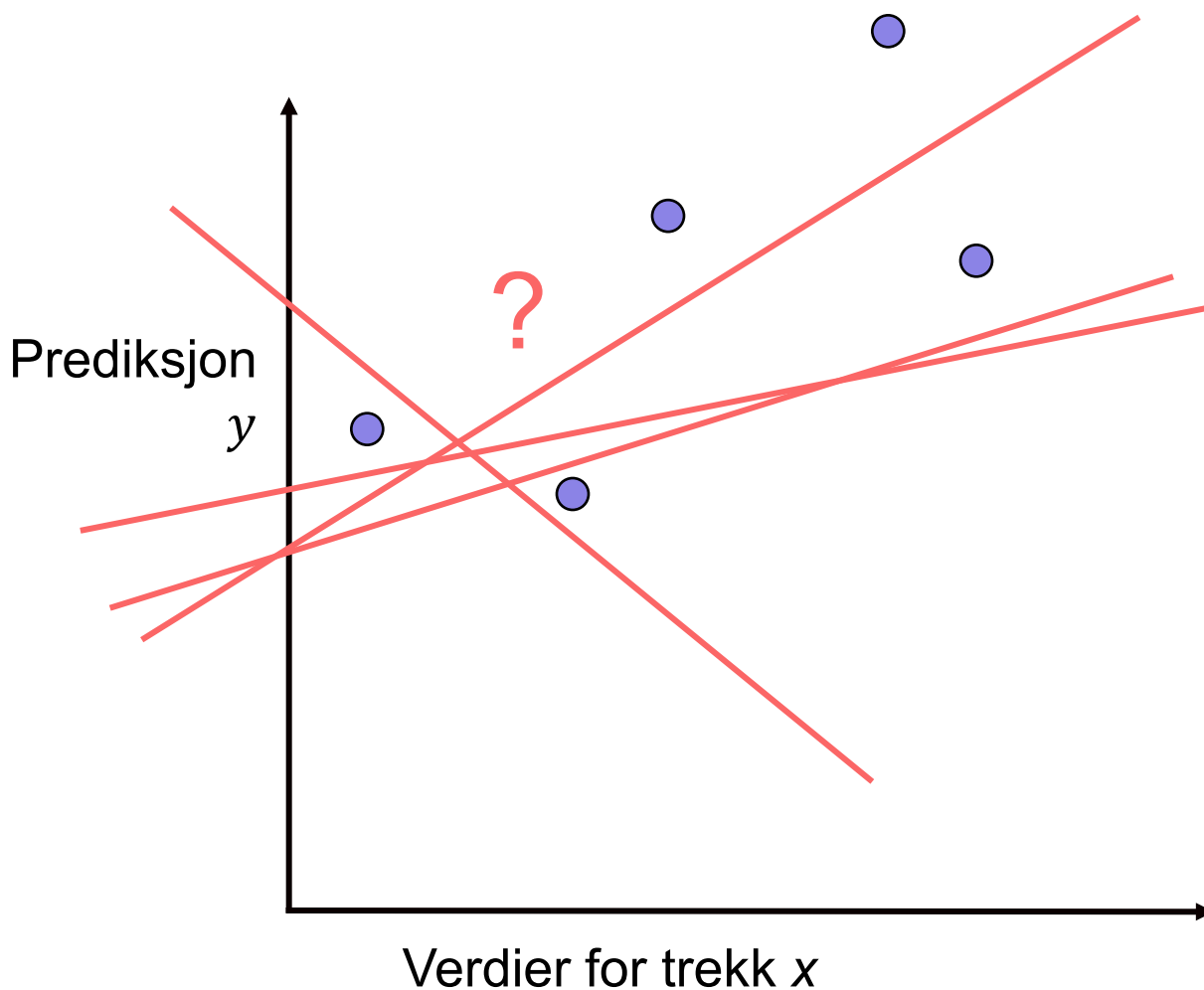
- $\hat{y}^{(k)} = \mathbf{w}^T \mathbf{x}^{(k)} + b$
- $\text{MSE} = \frac{1}{n} \sum_{k=1}^n (\hat{y}^{(k)} - y^{(k)})^2$

Læring

Intuisjon: læring handler om å finne verdiene for $w_1, \dots, w_p$ og b som gjør at “feilene” mellom hva modellen forutsier og fasitverdiene i treningssettet blir så *små som mulig*



Optimering



- **Mål:** finne parametere for **w** og **b** som minimerer tapet (MSE) på treningssettet
- Men hvordan går vi fram? Det er jo uendelige mange valg for **w** og **b**, og vi kan ikke teste ut alle!
- Heldigvis finnes det *optimeringsmetoder* som kan brukes til å estimere gode verdier for parameterne **w** og **b**, gitt en tapsfunksjon og et treningssett

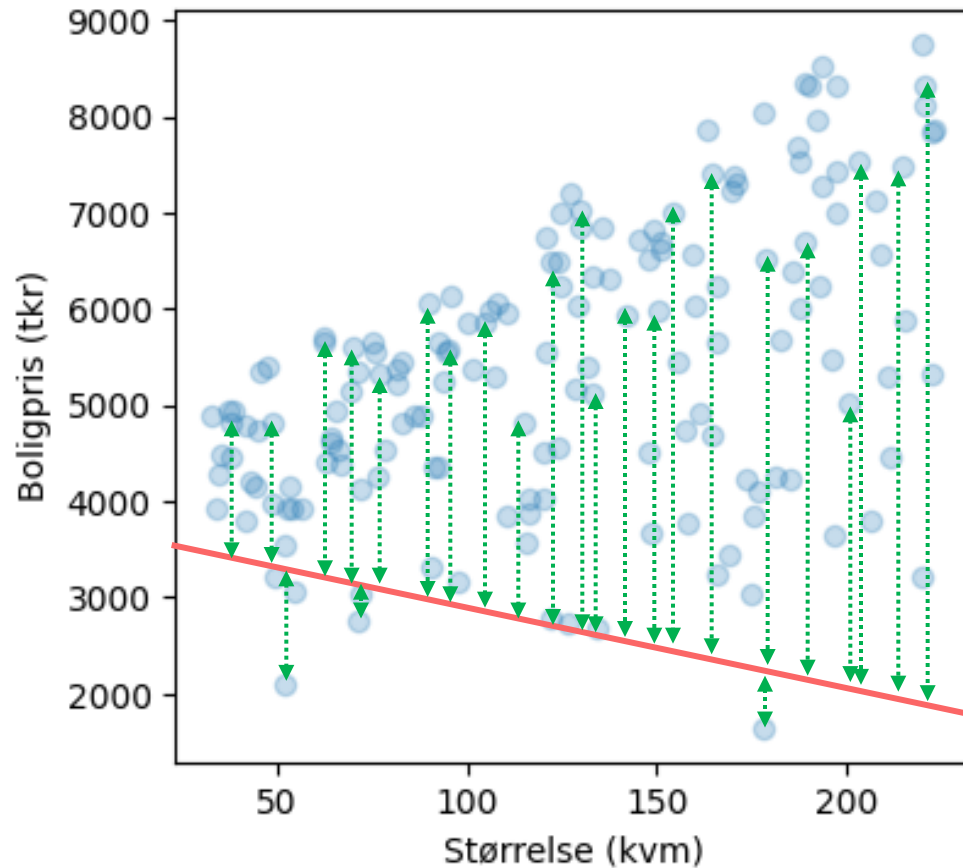
Optimering

- Optimeringsalgoritmer fungerer ofte iterativt:
 - Vi starter med en «dum» løsning for parameterne som skal estimeres
 - Og vi gradvis forbedrer denne inntil tapet ikke lenger lar seg redusere
- Den meste kjente optimeringsmetoden er **gradientnedstigning** (*gradient descent* eller GD)
 - De fleste optimeringsalgoritmer i maskinlæring i dag er varianter av GD
- Vi skal gå gjennom *intuisjonen* bak gradientnedstigning
 - Uten matematiske detaljer (som krever å forstå partiellderiverte)

Merk: Akkurat for lineær regresjon er det også mulig å estimere parameterne «analytisk», altså helt uten optimering. I dette kurset vil vi kun fokusere på optimeringsmetoder, av 2 grunner:

1. Analytiske løsninger sliter med å skalere når antallet trekk blir stort, som ofte er tilfellen i maskinlæring,
2. Disse løsningene kan dessuten ikke brukes for mer avanserte ML-modeller (slik som nevrale nettverk).

Optimering



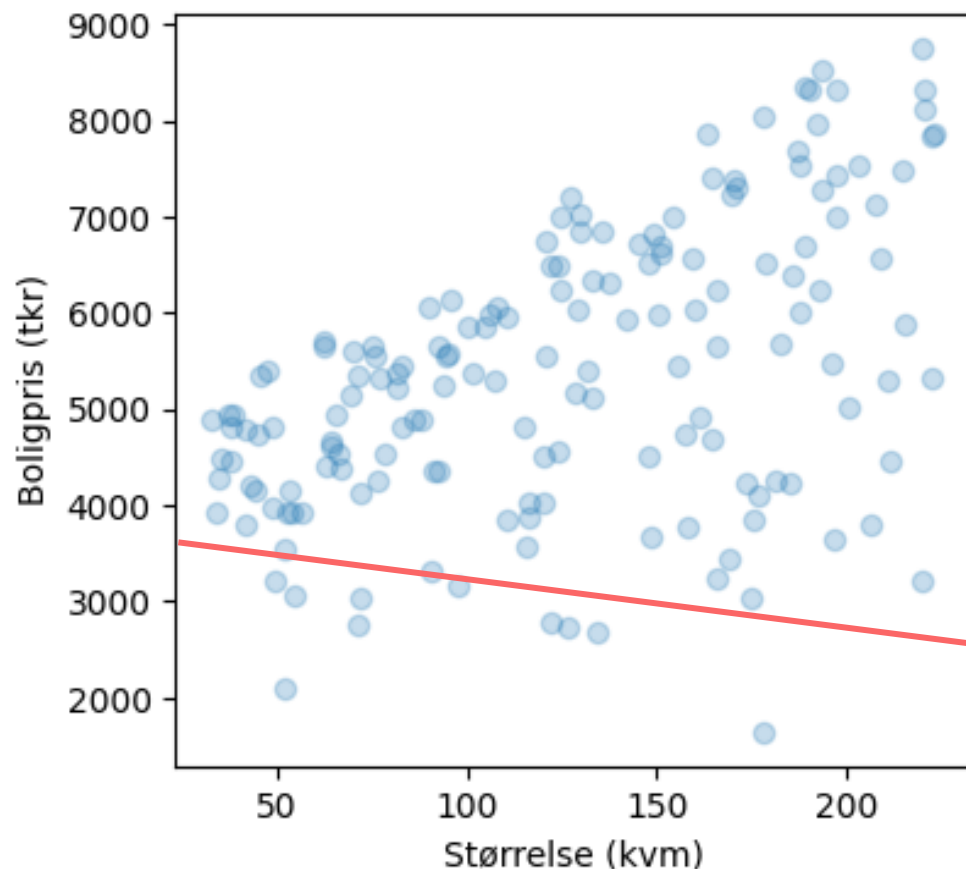
Mål: finne parametere for $\mathbf{w}$ og b som minimerer tapet (MSE) på treningssettet

1. Vi starter med tilfeldige verdier

$$\mathbf{w} \leftarrow \mathbf{w}^{(0)} \text{ og } b \leftarrow b^{(0)}$$

... som gir oss et ganske høyt tap

Optimering



Mål: finne parametere for **w** og **b** som minimerer tapet (MSE) på treningssettet

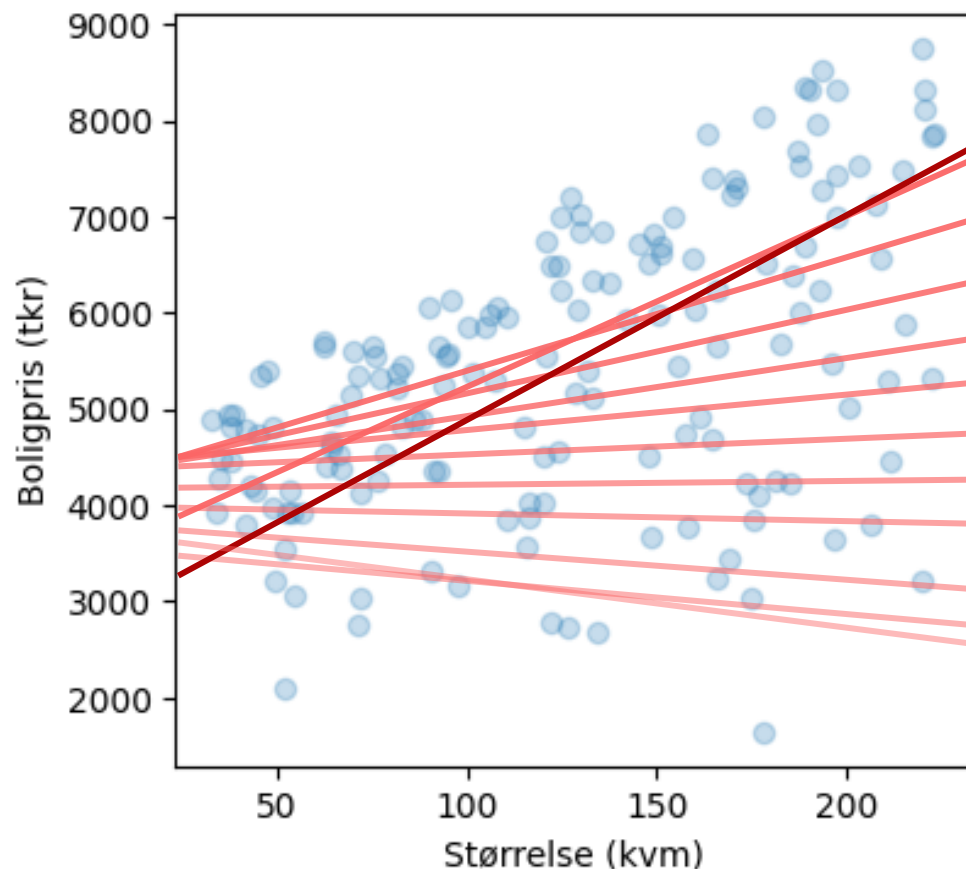
1. Vi starter med tilfeldige verdier
 $\mathbf{w} \leftarrow \mathbf{w}^{(0)}$ og $b \leftarrow b^{(0)}$
... som gir oss et ganske høyt tap
2. Vi justerer deretter **w** og **b** slik at tapet går (litt) nedover. Dette gjør vi ved å følge retningen hvor tapet synker *raskest*

For dere som har hatt litt mer avansert kalkulus på videregående: teknisk sett regner vi ut **gradienten**

$$\left[\frac{\partial \text{MSE}}{\partial w_1} \quad \frac{\partial \text{MSE}}{\partial w_2} \quad \cdots \quad \frac{\partial \text{MSE}}{\partial b} \right]$$

(men ikke stress, gradienter er ikke en del av pensumet)

Optimering

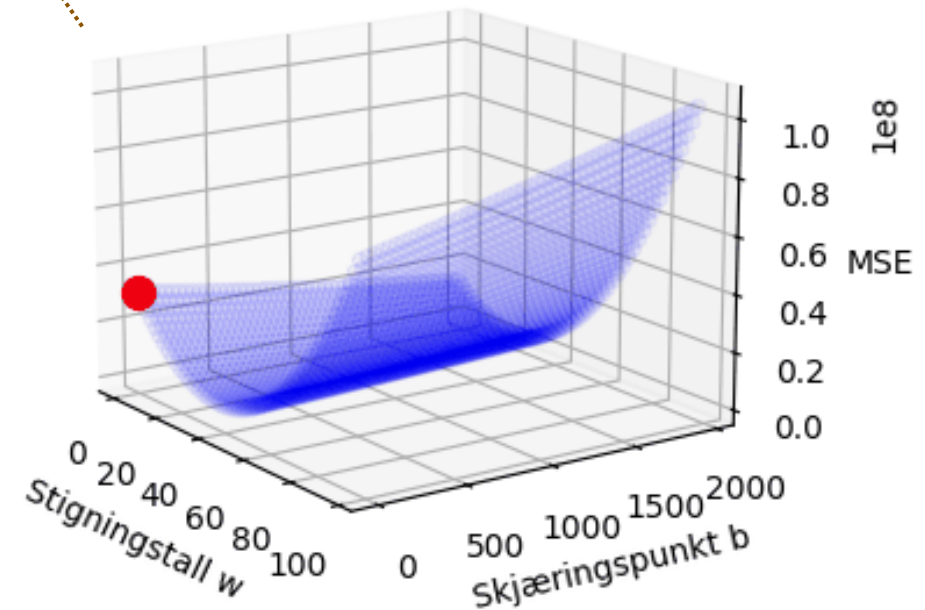
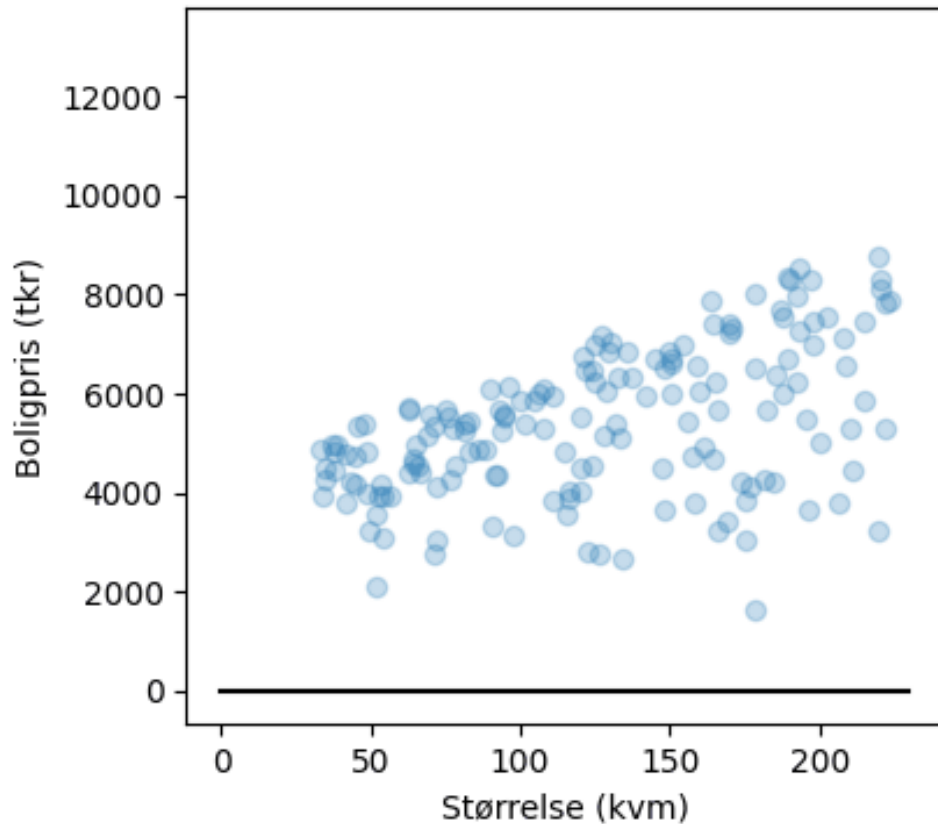


Mål: finne parametere for $\mathbf{w}$ og b som minimerer tapet (MSE) på treningssettet

1. Vi starter med tilfeldige verdier
 $\mathbf{w} \leftarrow \mathbf{w}^{(0)}$ og $b \leftarrow b^{(0)}$
... som gir oss et ganske høyt tap
2. Vi justerer deretter $\mathbf{w}$ og b slik at tapet går (litt) nedover. Dette gjør vi ved å følge retningen hvor tapet synker *raskest*
3. Vi fortsetter med små justeringer, steg etter steg, inntil tapet stabiliserer seg

Optimering

Vi kan tegne planen som viser det gjennomsnittlige tapet MSE på treningssettet avhengig av de to parameterne w og b . For hvert steg følger vi retningen hvor tapet synker raskest (ut fra gradienten), inntil vi når bunnpunktet.



Vi kan selv velge *læringsraten*, altså størrelsen på oppdateringen som utføres for hver iterasjon.

Optimering

- For større datamengder er det vanskelig å beregne gradienten for hele treningssettet
- I stedet beregner vi gradienten for små datamengder av gangen - såkalte «*mini-batch*», og oppdatere parameterne etter hver mini-batch

= «mini-batch» **stokastisk**
gradientnedstigning

(*stochastic gradient descent*,
eller SGD)

Treningsprosedyren blir altså:

1. Start med tilfeldige verdier for parameterne ($\mathbf{w}$, b)

Gjenta:

2. Ta en *mini-batch* $\mathcal{B}$ av treningsdata
 3. Beregn hvor mye modellen tar feil på $\mathcal{B}$ (MSE eller lignende)
 4. Beregn *gradienten* av dette tapet med hensyn på parameterne
 5. Juster parameterne et lite steg i retning hvor tapet synker raskest
- ... inntil tapet ikke lenger går ned

Merk: Vi kaller metoden “stokastisk” fordi hver oppdatering utføres på et lite, *tilfeldig* utvalg av treningssettet

Plan

- Modellen
- Læring
- **Prediksjon**
- Praktiske detaljer
- Oppsummering

Prediksjon

- Når vi har estimert en lineær regresjonsmodell kan vi enkelt bruke den på nye input $\mathbf{x} = [x_1 \quad x_2 \quad \dots \quad x_p]$:

$$y = \sum_{i=1}^p w_i x_i + b$$

Husk at p er antall *inputtrekk*

- En vanlig regresjonsmodell gir oss bare ett prediksjonstall
 - For eksempel at boligprisen blir 8.6 millioner kr
- Men det finnes mer avanserte versjoner som gir oss en hel *sannsynlighetsfordeling* over mulige outputverdier
 - Slik at vi kan si at prisen blir f.eks. mellom 7.8 og 9.3 mkr. med 95% sikkerhet

Prediksjon

Vi kan skrive n datapunkter i en matriseforamt:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_p^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_p^{(2)} \\ \dots & \dots & \dots & \dots \\ x_1^{(n)} & x_2^{(n)} & \dots & x_p^{(n)} \end{bmatrix}$$

p trekk per input

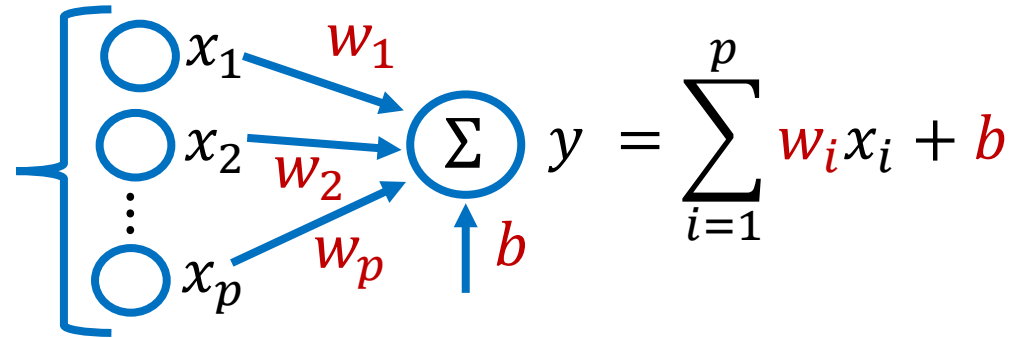
n datapunkter

Og vi kan beregne outputverdiene for alle punktene i ett steg:

$$\mathbf{y} = \mathbf{X} \mathbf{w} + \mathbf{b}$$

Multiplikasjon av en *matrise* og en *vektor* – hvis du ikke husker helt hvordan matriseregning fungerer, ikke stress, vi kommer til å gå gjennom det neste uke!

Prediksjon



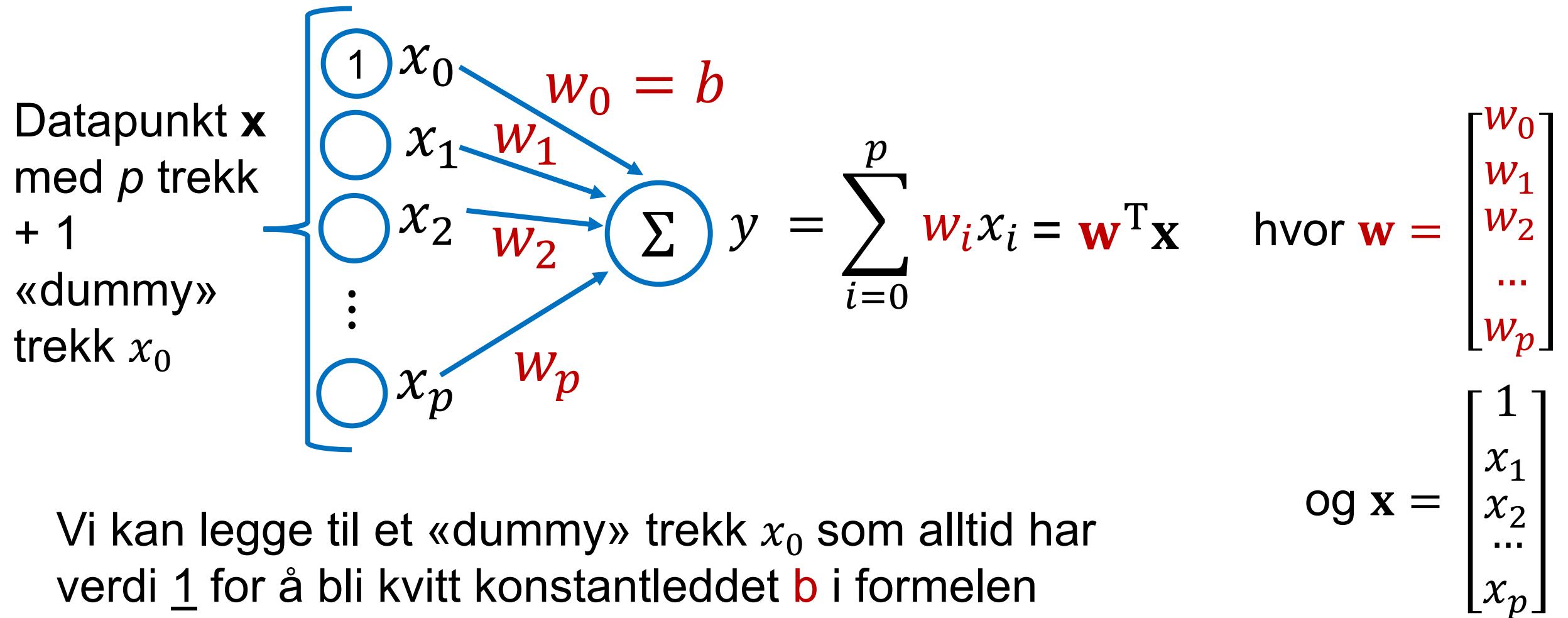
En vesentlig fordel av lineær regresjon er at modellen er *forklarbar* – vi kan forstå hvordan inputtrekkene “bidrar” til outputverdien:

- Hvis w_i er > 0 vil større verdier for x_i øke output y
- Hvis w_i er < 0 vil større verdier for x_i minske output y
- Hvis w_i er ≈ 0 vil x_i (omtrent) ikke påvirke y
- Og *større* (positive eller negative) vektorer $\rightarrow$ *større* påvirkning

Plan

- Modellen
- Læring
- Prediksjon
- **Praktiske detaljer**
- Oppsummering

Alternativ formulering



Vi kan legge til et «dummy» trekk x_0 som alltid har verdi 1 for å bli kvitt konstantleddet b i formelen

Merk: Dette er bare en alternativ notasjon - den endrer ikke selve modellen. Dere står fritt til å bruke standardnotasjonen med et eksplisitt konstantledd eller dette alternativet.

Trekkene

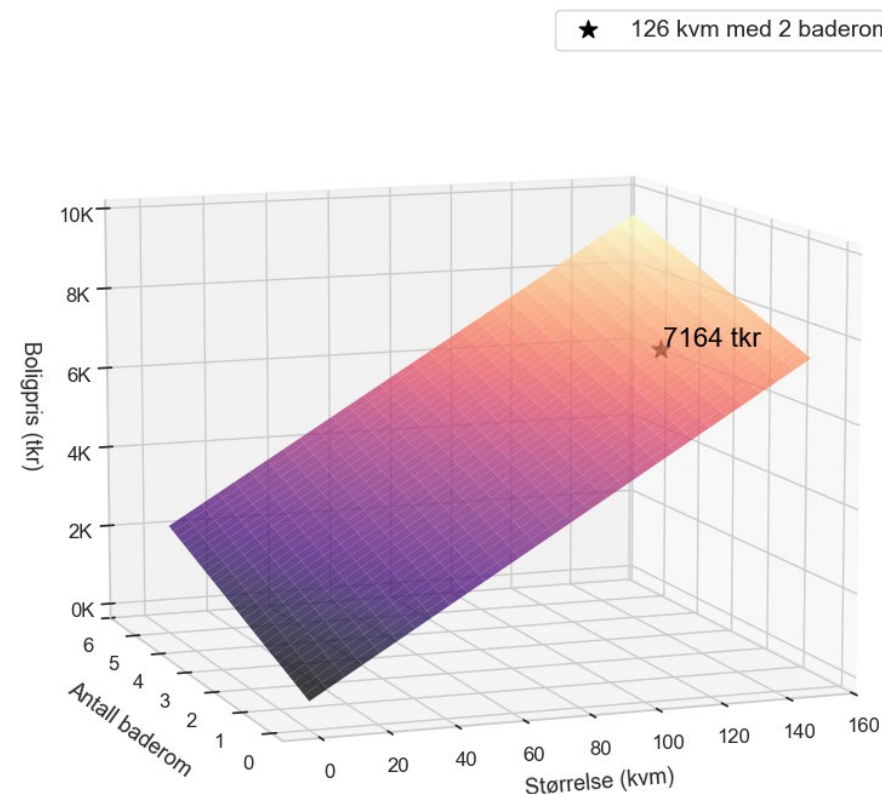
- Inntil nå har vi brukt trekkene “as is”, uten forbehandling
- Ofte en dårlig idé når trekkene har veldig forskjellige skalaer, slik som antall kvm og bad i eksempelet vårt

→ Store tallverdier bør *normaliseres*

- Kategoriske trekk (f.eks. «kjønn» eller «yrke») bør også *konverteres til tall*

Hvordan dettes gjøres konkret (og mye mer!) skal vi snakke om i forelesningen «ML i praksis»

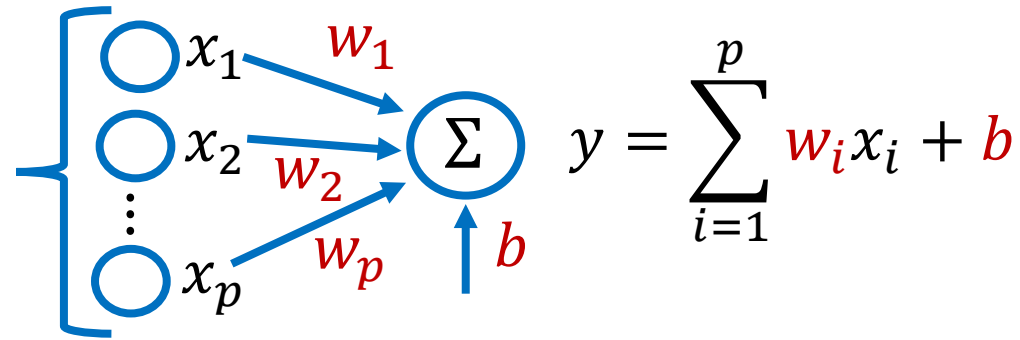
Hvorfor? Fordi trekk med store tallverdier risikerer da å “dominere” modellen



Plan

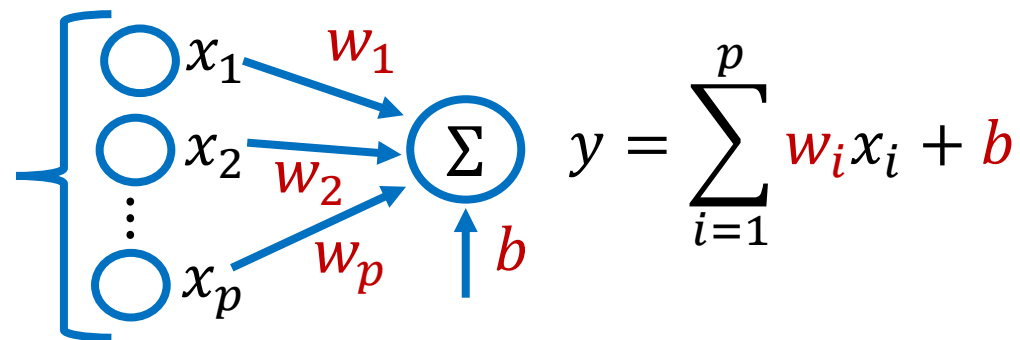
- Modellen
- Læring
- Prediksjon
- Praktiske detaljer
- **Oppsummering**

Oppsummering



- Lineær regresjon brukes til å predikere et tall y basert på et datapunkt $\mathbf{x}$ representert gjennom ulike trekk
- Modellen har parametere som må estimeres (læres) ut fra data: vektene $\mathbf{w} = [w_1 \quad w_2 \quad \dots \quad w_p]$ og konstantleddet b
- Læring = *leting* etter parameterverdier for $\mathbf{w}$ og b som gir minst mulig «tap» på et treningssett

Veien videre



Lineær regresjon utgjør grunnlaget for mange ML-modeller

- **Logistisk regresjon** (neste uke) er en utvidelse som kan brukes til predikere *klasser* i stedet for numeriske verdier
- **Nevrale nettverk** (om 2 uker) består i bunn av en rekke lineære regresjoner kombinert med ikke-lineære funksjoner
- Læringsprosessen vi har sett (minimering av tapet på et treningssett gjennom gradientnedstigning) er omtrent det samme for alle ML-modeller